

Document number: P0561R3

Date: 2017-11-11

Reply to: Geoff Romer <gromer@google.com>, Andrew Hunter <ahh@google.com>

Audience: Concurrency Study Group, Library Working Group

An RAII Interface for Deferred Reclamation

1. [Background](#)
2. [Design overview](#)
 1. [Read API](#)
 2. [Update API](#)
 3. [Clean shutdown](#)
 4. [Implementability](#)
3. [Open questions](#)
4. [Proposed wording](#)
5. [Revision history](#)
6. [Acknowledgements](#)

Background

For purposes of this paper, *deferred reclamation* refers to a pattern of concurrent data sharing with two components: *readers* access the data while holding reader locks, which guarantee that the data will remain live while the lock is held. Meanwhile one or more *updaters* update the data by replacing it with a newly-allocated value. All subsequent readers will see the new value, but the old value is not destroyed until all readers accessing it have released their locks. Readers never block the updater or other readers, and the updater never blocks readers. Updates are inherently costly, because they require allocating and constructing new data values, so they are expected to be rare compared to reads.

This pattern can be implemented in several ways, including reference counting, RCU, and hazard pointers. Several of these have been proposed for standardization (see e.g. [P0233R3](#), [P0279R1](#), and [P0461R1](#)), but the proposals have so far exposed these techniques through fairly low-level APIs.

In this paper, we will propose a high-level RAII API for deferred reclamation, which emphasizes safety and usability rather than fidelity to low-level primitives, but still permits highly efficient implementation. This proposal is based on an API which is used internally at Google, and our experience with it demonstrates the value of providing a high-level API: in our codebase we provide both a high-level API like the one proposed here, and a more bare-metal RCU API comparable to [P0461](#), and while the high-level API has over 200 users, the low-level API has one.

This proposal is intended to complement, rather than conflict with, proposals for low-level APIs for RCU, hazard pointers, etc, and it can be standardized independently of whether and when we standardize those other proposals.

Design overview

We begin with a simple example of how this API can be used: a `Server` class which handles requests using some config data, and can receive new versions of that config data (e.g. from a thread which polls the filesystem). Each request is handled using the latest config data available when the request is handled (i.e. updates do not affect requests already in flight). No synchronization is required between any of these operations.

```
class Server {
public:

    void SetConfig(Config new_config) {
        config_.update(std::make_unique<const Config>(std::move(new_config)));
    }

    void HandleRequest() {
        snapshot_ptr<const Config> config = config_.get_snapshot();

        // Use `config` like a unique_ptr<const Config>
    }

private:
    cell<Config> config_;
};
```

The centerpiece of this proposal is the `cell<T>` template, a wrapper which is either empty or holds a single object of type τ (the name is intended to suggest a storage cell, but we expect it to be bikeshedded). Rather than providing direct access to the stored object, it allows the user to obtain a *snapshot* of the current state of the object:

```
template <typename T, typename Alloc = allocator<T>>
class basic_cell {
public:
    // Not copyable or movable
    basic_cell(basic_cell&&) = delete;
    basic_cell& operator=(basic_cell&&) = delete;
    basic_cell(const basic_cell&) = delete;
    basic_cell& operator=(const basic_cell&) = delete;

    basic_cell(nullptr_t = nullptr, const Alloc& alloc = Alloc());
    basic_cell(std::unique_ptr<T> ptr, const Alloc& alloc = Alloc());

    void update(nullptr_t);
    void update(unique_ptr<T> ptr);

    bool try_update(const snapshot_ptr<T>& expected, std::unique_ptr<T>&& desired);

    snapshot_ptr<T> get_snapshot() const;
};

template <typename T>
using cell = basic_cell<see below>;

template <typename T>
class snapshot_ptr {
public:
    // Move only
    snapshot_ptr(snapshot_ptr&&) noexcept;
    snapshot_ptr& operator=(snapshot_ptr&&) noexcept;
    snapshot_ptr(const snapshot_ptr&) = delete;
    snapshot_ptr& operator=(const snapshot_ptr&) = delete;

    snapshot_ptr(nullptr_t = nullptr);

    // Converting operations, enabled if U* is convertible to T*
    template <typename U>
    snapshot_ptr(snapshot_ptr<U>&& rhs) noexcept;
    template <typename U>
    snapshot_ptr& operator=(snapshot_ptr<U>&& rhs) noexcept;

    T* get() const noexcept;
    T& operator*() const;
    T* operator->() const noexcept;

    explicit operator bool() const noexcept;

    void swap(snapshot_ptr& other);
};

template <typename T>
void swap(snapshot_ptr<T>& lhs, snapshot_ptr<T>& rhs);

template <typename T>
bool operator==(const snapshot_ptr<T>& lhs, const snapshot_ptr<T>& rhs);
// Similar overloads for !=, <, >, <=, and >=, and mixed
// comparison with nullptr_t.

template <typename T>
struct hash<snapshot_ptr<T>>;
```

As you can see, `cell` is actually an alias template that refers to `basic_cell`; we intend most users to use `cell` most of the time, although `basic_cell` is more fundamental. The problem with `basic_cell` is that, like so many other things in C++, it has the wrong default: a `basic_cell<std::string>` enables multiple threads to share *mutable* access to a `std::string` object, which is an open invitation to data races. Users can make the shared data immutable, but they have to opt into it by adding `const` to the type. We could add the `const` in the library, but then users would have no way to opt out if they genuinely do want to share mutable data (this is a reasonable thing to want, if the data has a race-free type).

This library is intended for routine use by non-expert programmers, so in our view safety *must* be the default, not something users have to opt into. Consequently, we provide a fully general but less safe API under the slightly more awkward name `basic_cell`, while reserving the name `cell` for a safe-by-default alias.

Specifically, `cell<T>` is usually an alias for `basic_cell<const T>`, but if the trait `is_race_free_v<T>` is true, signifying that τ can be

mutated concurrently without races, then `cell<T>` will be an alias for `basic_cell<T>`. Thus, `cell` exposes the most powerful interface that can be provided safely, while `basic_cell` provides users an opt-out, such as for cases where `T` is not inherently race-free, but the user will ensure it is used in a race-free manner. `basic_cell` thus acts as a marker of potentially unsafe code that warrants closer scrutiny, much like e.g. `const_cast`.

The major drawback we see in this approach is that it means that constness is somewhat less predictable: if `c` is a `cell<T>`, `c.get_snapshot()` might return a `snapshot_ptr<T>` or a `snapshot_ptr<const T>`, depending on `T`. We don't expect this to be a serious problem, because `const T` will be both the most common case by far, and the safe choice if the user is uncertain (`snapshot_ptr<T>` implicitly converts to `snapshot_ptr<const T>`). The problem can also be largely avoided through judicious use of `auto`.

Alternative approach: we could unconditionally define `cell<T>` as an alias for `basic_cell<const T>`. This would be substantially simpler, but `basic_cell` would not be able to act as a marker of code that requires close scrutiny, since many if not most uses of it (e.g. `basic_cell<atomic<int>>`) would be safe by construction. That said, any use of `basic_cell<T>` would still warrant some scrutiny, since shared mutable data usually carries a risk of race conditions, even if it is immune to data races.

Other than construction and destruction, all operations on `basic_cell` behave as atomic operations for purposes of determining a data race. One noteworthy consequence of this is that `basic_cell` is not movable, because there are plausible extensions of this design (e.g. to support user-supplied RCU domains) under which we believe that move assignment cannot be made atomic without degrading the performance of `get_snapshot`.

`cell`'s destructor does not require all outstanding `snapshot_ptr`s to be destroyed first, nor wait for them to be destroyed. This is motivated by the principle that concurrency APIs should strive to avoid coupling between client threads that isn't mediated by the API; concurrency APIs should solve thread coordination problems, not create new ones. That said, destruction of the `cell` must already be coordinated with the threads that actually read it, so also coordinating with the threads that hold `snapshot_ptr`s to it may not be much additional burden (particularly since `snapshot_ptr`s cannot be passed across threads).

Some deferred reclamation libraries are built around the concept of "domains", which can be used to isolate unrelated operations from each other (for example with RCU, long-lived reader locks can delay all reclamation within a domain, but do not affect other domains). This proposal does not include explicit support for domains, so effectively all users of this library would share a single global domain. So far our experience has not shown this to be a problem. If necessary domain support can be added, by adding the domain as a constructor parameter of `cell` (with a default value, so client code can ignore domains if it chooses), but it is difficult to see how to do so without exposing implementation details (e.g. RCU vs. hazard pointers).

Read API

`snapshot_ptr<T>`'s API is closely modeled on `unique_ptr<T>`, and indeed it could almost be implemented as an alias for `unique_ptr` with a custom deleter, except that we don't want to expose operations such as `release()` or `get_deleter()` that could violate API invariants or leak implementation details.

A `snapshot_ptr` is either null, or points to a live object of type `T`, and it is only null if constructed from `null_ptr`, moved-from, or is the result of invoking `get_snapshot()` on an empty `basic_cell` (in particular, a `snapshot_ptr` cannot spontaneously become null due to the actions of other threads). The guarantee that the object is live means that calling `get_snapshot()` is equivalent to acquiring a reader lock, and destroying the resulting `snapshot_ptr` is equivalent to releasing the reader lock.

In a high-quality implementation, all operations on a `snapshot_ptr` are non-blocking.

We require the user to destroy a `snapshot_ptr` in the same thread where it was obtained, so that this library can be implemented in terms of libraries that require reader lock acquire/release operations to happen on the same thread. Note that `unique_lock` implicitly imposes the same requirement, so this is not an unprecedented restriction. There are plausible use cases for transferring a `snapshot_ptr` across threads, and some RCU implementations can support it efficiently, but based on SG1 discussions we think it's safer to start with the more restrictive API, and broaden it later as needed.

The `const` semantics of `get_snapshot()` merit closer scrutiny. The proposed API permits users who have only `const` access to a `basic_cell<T>` to obtain non-`const` access to the underlying `T`. This is similar to the "shallow `const`" semantics of pointers, but unlike the "deep `const`" semantics of other wrapper types such as `optional`. In essence, the problem is that this library naturally supports three distinct levels of access (read-only, read-write, and read-write-update), but the `const` system can only express two. Our intuition (which SG1 in Kona generally seemed to share) is that the writer/updater distinction is more fundamental than the reader/writer distinction, so `const` should capture the former rather than the latter, but it's a close call.

We had considered providing `snapshot_ptr` with an aliasing constructor comparable to the one for `shared_ptr`:

```
template <typename U>
snapshot_ptr(snapshot_ptr<U>&& other, T* ptr);
```

This would enable the user, given a `snapshot_ptr` to an object, to construct a `snapshot_ptr` to one of its members. However, it would mean we could no longer guarantee that a `snapshot_ptr` is either null or points to a live object. SG1's consensus in Kona was to omit this feature, and we agree: we shouldn't give up that guarantee without a compelling use case.

Previous versions of this paper proposed that `snapshot_ptr<T>` rvalues be convertible to `shared_ptr<T>`, by analogy with `unique_ptr<T>`. However, this interface would require the user to ensure that the last copy of the resulting `shared_ptr` is destroyed on the same thread where the `snapshot_ptr` was created. This would be difficult to ensure in general, especially since the `shared_ptr`s carrying this requirement would be indistinguishable from any other `shared_ptr`. At the Toronto meeting, SG1 had no consensus to provide this conversion, so we have removed it. Peter Dimov points out that users who need to share ownership of a `snapshot_ptr` can do so [fairly easily](#) without this conversion.

Update API

The update side is more complex. It consists of two parallel sets of overloads, constructors and `update()`, which respectively initialize the `cell` with a given value, and update the `cell` to store a given value. `update()` does not necessarily wait for the old data value to be destroyed, although it may wait for other update operations. In addition, we provide a `try_update()` operation, which functions as a compare-and-swap, allowing us to support multiple unsynchronized updaters even when the new value depends on the previous value.

The constructor and `update()` overload taking `nullptr_t` respectively initialize and set the `cell` to the empty state. The fact that a `cell` can be empty is in some ways unfortunate, since it's generally more difficult to reason about types with an empty or null state, and users could always say `cell<optional<T>>` if they explicitly want an empty state. However, given that `snapshot_ptr` must have a null state in order to be movable, eliminating the empty state would not simplify user code much. Furthermore, forbidding the empty state when we support initialization from a nullable type would actually complicate the API.

The constructor and `update()` overload that accept a `unique_ptr<T>` `ptr` take ownership of it, and respectively initialize and set the current value of the cell to `*ptr`.

`try_update()` compares `expected` with the current value of the `cell` (i.e. the value that `get_snapshot()` would currently produce). If they are equal, it sets the current value of the cell to `desired` (setting `desired` to null in the process) and returns `true`. Otherwise, it returns `false` and leaves `desired` unmodified (spurious failures are also permitted, to maximize implementation flexibility). The execution of `try_update()` is atomic in both cases. Note that unlike other compare-and-swap operations, `try_update()` does not update `expected` on failure, because such an update could be costly and clients will not always need it. Clients who do can simply call `get_snapshot()` explicitly.

Internally, `cell` must maintain some sort of data structure to hold its previous values until it can destroy them, and sometimes this will require allocating memory. In [revision 0](#) of this paper, we discussed a possible mechanism by which the user could consolidate those allocations with their own allocation of the `T` data. However, this would effectively couple the library to a particular implementation, and greatly complicate the interface. Furthermore, its value is questionable, because those allocations can be made rare and small in normal usage (when `snapshot_ptr`s are destroyed within bounded time). In Kona, the SG1 consensus (which we agree with) was that such a mechanism is not necessary.

It also bears mentioning that this library may need to impose some restrictions on the allocators it supports. In particular, it may need to require that the allocator's `pointer` type is a raw pointer, or can safely be converted to one, since the implementation layer is unlikely to be able to accept "fancy pointers".

We have opted not to provide an emplacement-style constructor or `update` function, for several reasons. First of all, it provides no additional functionality; it's syntactic sugar for `update(make_unique<T>(...))` that might on some implementations be slightly more efficient (if it can consolidate allocations a la `make_shared`). Second, it would not be fully general; sometimes users need to perform some sort of setup in the interval between constructing the object and publishing it. Finally, it conflicts with another possible feature, support for custom deleters.

Currently, `unique_ptr`s passed to this library must use `std::default_delete`, but it's natural to ask if we could support other deleters. There are two ways we could go about that: we could make the deleter a template parameter of `cell`, or of the individual methods. Parameterizing the individual methods would be more flexible, but it would require some sort of type erasure, which would risk bloating the `cell` object (which can currently be as small as `sizeof(T*)`), and/or degrading performance on the read path (which needs to be fast). Parameterizing the whole class avoids type erasure, but precludes us from supporting `emplace`-style operations, because there's no way for the library to know how to allocate and construct an object so that it can be cleaned up by an arbitrary deleter.

Given the uncertainties around both features, the conflict between them, and the lack of strong motivation to add them, we have opted to omit them both for the time being.

One noteworthy property of `update()` is that there is no explicit support for specifying in the `update()` call how the old value is

cleaned up, which we are told is required in some RCU use cases in the Linux kernel. It is possible for the user to approximate support for custom cleanup by using a custom destructor whose behavior is controlled via a side channel, but this is a workaround, and an awkward one at that. We've opted not to include this feature because use cases for it appear to be quite rare (our internal version of this API lacks this feature, and nobody has asked for it), and because it would substantially complicate the API. It would add an extra `update()` parameter which most users don't need, and which would break the symmetry between constructors and `update()` overloads. More fundamentally, it would raise difficult questions about the relationship between the user-supplied cleanup logic and the original deleter: does the cleanup logic run instead of the deleter, or before the deleter? Neither option seems very satisfactory.

Clean shutdown

In Toronto, the concern was raised that some clients may want to ensure that all retired `cell` values are reclaimed before the program terminates (at least during "normal" termination). To support this, a previous version of this paper proposed introducing a namespace-scope function `set_synchronize_cells_on_exit()` which, if called, ensures that this will take place. However, SG1 consensus was that this was not the right solution, and instead the space should be left open for a future paper, so we have removed that functionality from the proposal.

Implementability

This proposal is designed to permit implementation via RCU, hazard pointers, or reference counting (or even garbage collection, we suppose). It is also designed to permit implementations that perform reclamation on background threads (which can enable `update()` to be nonblocking and lock-free regardless of τ), as well as implementations that reclaim any eligible retired values during `update()` calls (which can ensure that `update()` is truly wait-free if $\sim\tau()$ is, and ensure a bound on the number of unreclaimed values). These two techniques appear to be mutually exclusive, and neither seems dramatically superior to the other: this API is not intended for cases where `update()` is a performance bottleneck, and in practice the number of retired but unreclaimed values should be tightly bounded in normal use, even if it is theoretically unbounded. Consequently, we propose to permit either implementation strategy, by not bounding the number of live old values and permitting `update()` to delete unreclaimed old values.

We tentatively propose to also allow the implementation to perform reclamation during `~snapshot_ptr`, because that's the most natural choice for reference counting, but we're concerned that this risks adding latency to the read path (e.g. if $\sim\tau$ is slow or blocking). The alternative would be to require reference-counting implementations to defer reclamation to a subsequent `update()` call or a separate thread, but that would probably be slower when τ is trivially destructible. We are opting not to impose this constraint because it will be easier to add later than to remove.

We do not intend to support the trivial implementation strategy of never performing any reclamation at all, but it is not yet clear if we will be able to disallow it without also disallowing other more reasonable implementation strategies. If we are not able to make this implementation nonconforming, we will non-normatively discourage it as strongly as possible.

This proposal cannot be implemented in terms of an RCU library that requires user code to periodically enter a "quiescent" state where no reader locks are held. We see no way to satisfy such a requirement in a general-purpose library, since it means that any use of the library, no matter how local, imposes constraints on the global structure of the threads that use it (even though the top-level thread code may otherwise be completely unaware of the library). This would be quite onerous to comply with, and likely a source of bugs. Neither [Userspace RCU](#), Google's internal RCU implementation, nor [P0233R2](#)'s hazard pointer API impose this requirement, so omitting it does not appear to be a major implementation burden.

This proposal also does not require user code to register and unregister threads with the library, for more or less the same reasons: it would cause local uses of the library to impose global constraints on the program, creating an unacceptable usability and safety burden. [P0233R2](#) and Google's internal RCU do not impose this requirement, and Userspace RCU provides a library that does not (although at some performance cost). Furthermore, the standard library can satisfy this requirement if need be, without exposing users to it, by performing the necessary registration in `std::thread`.

A previous version of this paper stated that we did not think this API could be implemented in terms of hazard pointers, but that was an error. We are aware of no obstacles to implementing this library in terms of hazard pointers. However, we expect RCU to be the preferred implementation strategy in practice, because it can provide superior performance on the read side.

Open questions

SG1 has identified the following open questions, which they hope will be resolved through TS feedback and/or followup proposals, but do not block adoption in a TS:

- Should we define more precisely/normatively the requirements on types for which `is_race_free` is true? If so, how?
- Should `is_race_free` have a different name? If so, what?
- Should this library support use cases that cannot tolerate a hidden background thread for performing reclamation? If so,

how?

- Should this library support clean shutdown, and/or a way to wait for reclamation of retired values? If so, how?
- How should the wording/API be revised to support consume semantics?

Proposed wording

This proposal is targeted to the Concurrency TS. We expect the section structure and introductory material to evolve as other related proposals are added, but we are initially presenting it as a new top-level clause.

Deferred reclamation [concur.cell]

Deferred reclamation overview [concur.cell.overview]

This clause describes components that a C++ program can use to manage the lifetime of data that may be shared between threads. They can be used to keep objects alive while they are potentially being concurrently accessed, while ensuring those objects are destroyed once they are no longer accessible. [*Note*: these components are not restricted to multi-threaded programs, but can be useful in single-threaded programs as well — *end note*]

A variety of implementation techniques are possible, including RCU, hazard pointers, and atomic reference counting.

Header <cell> synopsis [concur.cell.synop]

```
namespace std {
    // ??, is_race_free trait
    template <class T> class is_race_free;
    template <class T> inline constexpr bool is_race_free_v
        = is_race_free<T>::value;

    // ??, Class template basic_cell
    template <class T, class Allocator = allocator<T>>
    class basic_cell;

    // ??, Alias template cell
    template <class T, class Allocator = allocator<T>>
    using cell = basic_cell<see below, Allocator>;

    // ??, Class template snapshot_ptr
    template <class T> class snapshot_ptr;

    // ??, snapshot_ptr specialized algorithms
    template <class T>
    void swap(snapshot_ptr<T>& lhs, snapshot_ptr<T>& rhs);

    template <class T, class U>
    bool operator==(const snapshot_ptr<T>& lhs, const snapshot_ptr<U>& rhs);
    template <class T, class U>
    bool operator!=(const snapshot_ptr<T>& lhs, const snapshot_ptr<U>& rhs);
    template <class T, class U>
    bool operator<(const snapshot_ptr<T>& lhs, const snapshot_ptr<U>& rhs);
    template <class T, class U>
    bool operator<=(const snapshot_ptr<T>& lhs, const snapshot_ptr<U>& rhs);
    template <class T, class U>
    bool operator>(const snapshot_ptr<T>& lhs, const snapshot_ptr<U>& rhs);
    template <class T, class U>
    bool operator>=(const snapshot_ptr<T>& lhs, const snapshot_ptr<U>& rhs);

    template <class T>
    bool operator==(nullptr_t, const snapshot_ptr<T>& rhs);
    template <class T>
    bool operator==(const snapshot_ptr<T>& lhs, nullptr_t);
    template <class T>
    bool operator!=(nullptr_t, const snapshot_ptr<T>& rhs);
    template <class T>
    bool operator!=(const snapshot_ptr<T>& lhs, nullptr_t);
    template <class T>
    bool operator<(nullptr_t, const snapshot_ptr<T>& rhs);
    template <class T>
    bool operator<(const snapshot_ptr<T>& lhs, nullptr_t);
    template <class T>
```

```

bool operator<=(nullptr_t, const snapshot_ptr<T>& rhs);
template <class T>
bool operator<=(const snapshot_ptr<T>& lhs, nullptr_t);
template <class T>
bool operator>(nullptr_t, const snapshot_ptr<T>& rhs);
template <class T>
bool operator>(const snapshot_ptr<T>& lhs, nullptr_t);
template <class T>
bool operator>=(nullptr_t, const snapshot_ptr<T>& rhs);
template <class T>
bool operator>=(const snapshot_ptr<T>& lhs, nullptr_t);

template <class T>
struct hash<snapshot_ptr<T>>;
}

```

is_race_free trait

```
template <class T> class is_race_free;
```

This template shall be a `UnaryTypeTrait` with a base characteristic of `true_type` or `false_type`.

The base characteristic is `true_type` when T is a specialization of `atomic<T>`.

The base characteristic is `false_type` when T is a user-defined type. This requirement does not apply to user-defined specializations of `is_race_free`.

[*Note*: This trait is used to disable certain safety measures that prevent mutation of T objects that may be accessible to other threads. Consequently, it should have a base characteristic of `true_type` only if T 's contract permits mutations that are concurrent with other operations. — *end note*]

Class template `basic_cell` [`concur.basic_cell`]

An object of type `basic_cell<T, Allocator>` represents a pointer to an object of type T , and provides operations to access and update the currently stored pointer. Updates are expected to be rare relative to accesses, so implementations should ensure that read-side operations (`get_snapshot()` and operations on `snapshot_ptr`) are non-blocking (and in particular don't reclaim synchronously) and as fast as possible. A `basic_cell` owns all pointers stored in it, but ensures that previous values are not reclaimed until they can no longer be accessed (hence the term "deferred reclamation").

```

namespace std {
    template <class T, class Allocator = see below>
    class basic_cell {
    public:
        using element_type = T;

        // ??, constructors
        constexpr basic_cell(nullptr_t = nullptr, const Allocator& a = Allocator());
        basic_cell(unique_ptr<T> ptr, const Allocator& a = Allocator());

        // ??, destructor
        ~basic_cell();

        // ??, update
        void update(nullptr_t);
        void update(unique_ptr<T> ptr);
        bool try_update(const snapshot_ptr<T>& expected, std::unique_ptr<T>&& desired);

        // ??, value access
        snapshot_ptr<T> get_snapshot() const;

        // Disable copy and move
        basic_cell(basic_cell&&) = delete;
        basic_cell& operator=(basic_cell&&) = delete;
        basic_cell(const basic_cell&) = delete;
        basic_cell& operator=(const basic_cell&) = delete;
    };
}

```

A `basic_cell`'s *value* consists of the pointer the user stored in it, but if two different `basic_cell` operations store

equal non-null pointers, the resulting values are considered to be distinct. [*Note*: In other words, `basic_cell` values are considered to be the same only if they are both null, or were caused by the same update operation. — *end note*]. Furthermore, if one of these operations does not happen after the reclamation of the value resulting from the other, the behavior is undefined. [*Note*: Consequently, non-equal values representing equal pointers are never concurrently live — *end note*]

For purposes of determining the existence of a data race, all member functions of `basic_cell` (other than construction and destruction) behave as atomic operations on the value of the `basic_cell` object.

All modifications to the value of a `basic_cell` occur in a particular total order, called the *modification order*, which is consistent with the happens-before partial order.

The default value of `Allocator` shall be a specialization of `std::allocator`. `Allocator` must satisfy the requirements of an allocator (20.5.3.5). For all types `U`, `allocator_traits<Allocator>::rebind_traits<U>::pointer` must be `U*`.

`basic_cell<T>` *reclaims* previous non-null values by invoking `default_delete<T>()` on them, but this reclamation is deferred until it can satisfy all the "synchronizes with" constraints specified in this subclause. When reclamation of a value would satisfy those constraints, the value is said to be *eligible* for reclamation.

[*Note*: There is no way to ensure that any given `cell` value is ever reclaimed, even at program termination, so `cell` may not be suitable for managing objects whose destructors have observable side effects. The implementation should ensure that all but a bounded number of values are reclaimed within a bounded amount of time after they are eligible for reclamation. — *end note*]

`basic_cell` constructors [concur.basic_cell.ctor]

```
constexpr basic_cell(nullptr_t = nullptr, const Allocator& a = Allocator());
```

Effects: Equivalent to `basic_cell(unique_ptr<T>(), a)`.

```
basic_cell(unique_ptr<T> ptr, const Allocator& a = Allocator());
```

Effects: Initializes the `basic_cell` with `ptr.get()` as its initial value. The `basic_cell` will use a copy of `a` to obtain memory, if necessary.

`basic_cell` destructor [concur.basic_cell.dtor]

```
~basic_cell()
```

Effects: May reclaim the current value of `*this` if it is eligible for reclamation, but will not block for it to become eligible.

Synchronization: If `*this` has a non-null value, the start of this operation synchronizes with the reclamation of the value.

`basic_cell` update operations [concur.basic_cell.update]

```
void update(nullptr_t);
```

Effects: equivalent to `update(unique_ptr<T>())`.

```
void update(unique_ptr<T> ptr);
```

Effects: Atomically sets the value of `*this` to `ptr.get()`. May then reclaim the previous value of `*this` (in the modification order), if it is eligible for reclamation, but will not block for it to become eligible.

Synchronizaton: The atomic portion of this operation synchronizes with reclamation of the previous value of `*this` (in the modification order).

```
bool try_update(const snapshot_ptr<T>& expected, std::unique_ptr<T>&& desired);
```

Effects: If `expected.get()` is equal to the current value of `*this`, then with high probability the value of `*this` will be set to `desired.release()` and the call will return `true`. Otherwise, the call will return `false` and have no other effect.

Synchronization: If the call returns `true`, it synchronizes with the reclamation of the value of `expected`.

Notes: This operation never causes reclamation.

basic_cell value access [concur.basic_cell.access]

```
snapshot_ptr<T> get_snapshot() const;
```

Returns: A snapshot_ptr containing the current value of *this.

Synchronization: The update() or try_update() call (if any) that caused *this to have its current value synchronizes with this operation.

Alias template cell [concur.cell.cell]

```
template <class T, class Allocator = allocator<T>>
using cell = basic_cell<see below, Allocator>;
```

If is_race_free_v<T> is true, this is an alias for basic_cell<T, Allocator>. Otherwise, it is an alias for basic_cell<const T, Allocator>.

[*Note:* As a result, for most non-pathological types τ , cell< $\tau\tau$ objects accessed through it. — end note]

Class template snapshot_ptr [concur.snapshot_ptr]

A snapshot_ptr is smart pointer that can represent a "snapshot" of the value of a basic_cell at a certain point in time. Every snapshot_ptr is guaranteed to either be null, or point to a live object of type τ , so holding a snapshot_ptr prevents the object it points to from being destroyed.

[*Note:* In some implementations, a long-lived snapshot_ptr can prevent reclamation of any basic_cell values (anywhere in the program) that weren't eligible for reclamation it was created, so user code should ensure that snapshot_ptrs have a bounded lifetime. — end note]

A snapshot_ptr behaves as an ordinary value type, like unique_ptr; it will not be accessed concurrently unless user code does so explicitly, and it has no protection against data races other than what is specified for the library generally (20.5.5.9).

```
namespace std {
    template <class T>
    class snapshot_ptr {
    public:
        // ??, snapshot_ptr constructors
        snapshot_ptr(nullptr_t = nullptr);
        snapshot_ptr(snapshot_ptr&& other) noexcept;
        template <class U>
        snapshot_ptr(snapshot_ptr<U>&& other) noexcept;

        // ??, snapshot_ptr destructor
        ~snapshot_ptr();

        // ??, snapshot_ptr assignment
        snapshot_ptr& operator=(snapshot_ptr&& other) noexcept;
        template <class U>
        snapshot_ptr& operator=(snapshot_ptr<U>&& other) noexcept;

        // ??, snapshot_ptr observers
        T* get() const noexcept;
        T& operator*() const;
        T* operator->() const noexcept;
        explicit operator bool() const noexcept;

        // ??, snapshot_ptr modifiers
        void reset(nullptr_t = nullptr) noexcept;
        void swap(snapshot_ptr& other) noexcept;

        // disable copy from lvalue
        snapshot_ptr(const snapshot_ptr&) = delete;
        snapshot_ptr& operator=(const snapshot_ptr&) = delete;
    };
}
```

snapshot_ptr objects take on the same values as basic_cell objects. The value of a snapshot_ptr will not change

except as explicitly specified below.

snapshot_ptr constructors [concur.snapshot_ptr.ctor]

```
snapshot_ptr(nullptr_t = nullptr);
```

Effects: Initializes *this with a null value.

```
snapshot_ptr(snapshot_ptr&& other) noexcept;  
template <class U>  
snapshot_ptr(snapshot_ptr<U>&& other) noexcept;
```

Remarks: The second overload shall not participate in overload resolution unless U* is implicitly convertible to T*.

Postconditions: other == nullptr, and *this has the value that other had prior to the operation.

snapshot_ptr destructor [concur.snapshot_ptr.dtor]

```
~snapshot_ptr()
```

Requires: If the value of *this is not null, then the get_snapshot() call that produced the value of *this occurred on the current thread.

Effects: May reclaim the value of *this if it is eligible for reclamation, but will not block for it to be eligible.

Synchronization: If the value of *this is not null, the start of this operation synchronizes with the reclamation of the value.

snapshot_ptr assignment [concur.snapshot_ptr.assign]

```
snapshot_ptr& operator=(snapshot_ptr&& other) noexcept;  
template <class U>  
snapshot_ptr& operator=(snapshot_ptr<U>&& other) noexcept;
```

Remarks: The second overload shall not participate in overload resolution unless U* is implicitly convertible to T*.

Effects: May reclaim the value that other had prior to the operation, if it is eligible for reclamation, but will not block for it to be eligible.

Postconditions: other == nullptr, and *this has the value that other had prior to the operation.

Returns: *this

Synchronization: If *this had a non-null value prior to the operation, the start of the operation synchronizes with the reclamation of the value.

snapshot_ptr observers [concur.snapshot_ptr.observers]

```
T* get() const noexcept;
```

Returns: The value of *this.

```
T& operator*() const;
```

Effects: Equivalent to return *get();.

```
T* operator->() const noexcept;
```

Requires: get() != nullptr.

Returns: get().

```
explicit operator bool() const noexcept;
```

Returns: get() != nullptr.

snapshot_ptr modifiers [concur.snapshot_ptr.modifiers]

```
void reset(nullptr_t = nullptr) noexcept;
```

Postconditions: *this contains a null value.

```
void swap(snapshot_ptr& other) noexcept;
```

Effects: Exchanges the values of *this and other.

snapshot_ptr specialized algorithms [concur.snapshot_ptr.alg]

```
template <class T>
void swap(snapshot_ptr<T>& lhs, snapshot_ptr<T>& rhs) noexcept;
```

Effects: Equivalent to lhs.swap(rhs).

```
template <class T, class U>
bool operator==(const snapshot_ptr<T>& lhs, const snapshot_ptr<U>& rhs);
```

Returns: True if lhs and rhs contain the same value, and false otherwise.

```
template <class T, class U>
bool operator!=(const snapshot_ptr<T>& lhs, const snapshot_ptr<U>& rhs);
```

Returns: !(lhs == rhs).

```
template <class T, class U>
bool operator<(const snapshot_ptr<T>& lhs, const snapshot_ptr<U>& rhs);
```

Effects: Equivalent to return less<common_type_t<T*, U*>>()(L, R);, where L is the value held by lhs and R is the value held by rhs.

```
template <class T, class U>
bool operator<=(const snapshot_ptr<T>& lhs, const snapshot_ptr<U>& rhs);
```

Returns: !(rhs < lhs).

```
template <class T, class U>
bool operator>(const snapshot_ptr<T>& lhs, const snapshot_ptr<U>& rhs);
```

Returns: rhs < lhs.

```
template <class T, class U>
bool operator>=(const snapshot_ptr<T>& lhs, const snapshot_ptr<U>& rhs);
```

Returns: !(lhs < rhs).

```
template <class T>
bool operator==(nullptr_t, const snapshot_ptr<T>& rhs);
template <class T>
bool operator!=(nullptr_t, const snapshot_ptr<T>& rhs);
template <class T>
bool operator<(nullptr_t, const snapshot_ptr<T>& rhs);
template <class T>
bool operator<=(nullptr_t, const snapshot_ptr<T>& rhs);
template <class T>
bool operator>(nullptr_t, const snapshot_ptr<T>& rhs);
template <class T>
bool operator>=(nullptr_t, const snapshot_ptr<T>& rhs);
```

Returns: snapshot_ptr<T>() OP rhs, where OP is the operator being defined.

```
template <class T>
bool operator==(const snapshot_ptr<T>& lhs, nullptr_t);
template <class T>
bool operator!=(const snapshot_ptr<T>& lhs, nullptr_t);
template <class T>
bool operator>(const snapshot_ptr<T>& lhs, nullptr_t);
template <class T>
bool operator<(const snapshot_ptr<T>& lhs, nullptr_t);
template <class T>
bool operator<=(const snapshot_ptr<T>& lhs, nullptr_t);
template <class T>
bool operator>=(const snapshot_ptr<T>& lhs, nullptr_t);
```

Returns: lhs *OP* snapshot_ptr<T>(), where *OP* is the operator being defined.

```
template <class T>
struct hash<snapshot_ptr<T>>;
```

The specialization is enabled ([unord.hash]).

Revision history

Since P0561R2:

- Removed support for clean shutdown.
- Centralized notes on performance as normative encouragement in the preface.
- Dropped incorrect use of dependency order.
- Explicitly targeted the Concurrency TS.
- Miscellaneous wording tweaks.

Since P0561R1:

- Dropped `shared_ptr` conversion.
- Added support for clean shutdown.
- Added `try_update()`.
- Added wording.

Since P0561R0:

- Introduced `basic_cell` and altered the role of `is_race_free`, in order to provide a per-instance (as well as per-type) opt-out of default thread-safety.
- `update()` calls are now guaranteed not to race with each other, because it simplifies the API: it's easier to remember that `cell` is always race-free than to try to keep track of which operations can race with which. As noted above, updates are expected to be relatively rare, so additional locking in `update()` should not matter, and in any event common implementations should be able to support this without locking.
- Added some discussion of why `cell` is not movable.
- Documented decision to omit an aliasing constructor.
- Required `snapshot_ptr`s to be destroyed in the same thread where they are created; see the main text for the rationale.
- Simplified discussion of allocation, and documented decision not to provide a mechanism like `cell_init`.
- Documented decision to omit emplacement-style operations and support for custom deleters.
- Documented decision that `const` access to `cell<T>` grants ability to mutate the `T`.
- Documented concerns with `use_count()` and `unique()` on `shared_ptr`s created from `snapshot_ptr`s.
- Dropped claim that this library cannot be implemented in terms of hazard pointers, which doesn't appear to be true.
- Added discussion of tradeoff between bounded space and fast `update()`.
- Added a usage example.

Acknowledgements

Thanks to Paul McKenney and Maged Michael for valuable feedback on drafts of this paper.